

25A – M5 Inventory, Costing, Repack & Loss: Implementation Guide

Everything the M5 developer needs: the inventory schema as DDL, seed data, package layout, the API slice, handler specifications with pseudocode for the ledger core (receipt, sale deduction with oversell, weighted average, count to adjustment, write-off posting, repack and reversal, availability), the consumers and jobs, the lot snapshot and the till bundle contracts for count, write-off draft, repack and transfer receipt, screens, tests and tickets.

Field	Value
Document	25A – implementation guide for 25 M5 Inventory, Costing, Repack & Loss v0.1
Version	v0.1 – for the M5 developer; sections 7.3 and 7.4 are the contract with the till track
Date	9 September 2026
Prerequisite	Sprint 0; M1–M4 contracts frozen (M4's grn.confirmed.v1, claim.approved.v1, transfer_request.approved.v1, delivery_note.issued.v1 payloads). M6's receipt bundle shape is taken from doc 32 §3.1 and doc 18 document_line.
Estimated effort	About 27 developer-days for one full-stack developer
Restated content	Marked <i>restated from doc X §Y</i> .

Read this first. Build the ledger core first: `LedgerService.post(movements)` is the one place stock changes, and everything else – receipts, sales, counts, write-offs, repacks, transfers, opening balances – is a caller that assembles movements and cites a document. Get the property test 'lot balance equals movement sum under any interleaving' green on day two, then never touch `qty_on_hand` from anywhere else.

1. What you are building

Restated from doc 25 §1. Where stock is and what it cost, by location, batch and condition; the append-only ledger; weighted-average costing per entity; availability; counts, adjustments, write-offs, repacks, transfers and opening balances as documents; expiry tasks; oversell exceptions.

You build	You do not build
Schema inventory: stock_lot, stock_movement, entity_sku_cost, repack_recipe, count_task, expiry_task, pick_list, loss_category, variance_threshold, loss_tolerance, and the document extensions for CNT, ADJ, WOF, RPK, XFR, OPB	Batch identity (M2), prices (M3), GRN and claims (M4), receipts (M6)
The ledger service; 5 internal apply commands; 22 user commands; 12 queries; 9 event families; 4 consumers; 4 jobs; the lot snapshot contributor	The till screens (M6 renders count, write-off, repack, transfer receipt from your specification)
Eleven web screens	–

2. What you inherit

From	You use	How
19A §7 document base	DocumentService.issue, DocumentLinks (REVERSES for repack reversal and transfer cancel), attachments	All M5 documents
19A §8 sync	Bundle ingestion hooks for CNT, WOF (draft), RPK, XFR receipt; SnapshotContributor; change log	Sections 7.3, 7.4
19A §3 permissions	Sod.assertDistinct, role limits for value bands	Adjustment and write-off approval
19A §4–5, 11, 12	Audit, events, config, scheduled jobs	As every module
Shared engine	Money, Quantity (scale 3 quantities, scale 4 costs)	Never re-implement rounding
M1 query	Locations of an entity; location type; single-staff flag (size band S, or a per-location config)	Witness rule; series scope
M2 api/query	RegisterBatch (repack outputs), GetSku flags and conversions, batch identity, alias	Repack, counts, re-pointing
M3 query	ResolveTradePrice is not needed here (M4 gives unit cost on the GRN event)	–
M4 events	grn.confirmed/reversed, claim.approved, drop.undelivered, delivery_note.issued, transfer_request.approved	Section 7.1
M6 bundles	receipt.issued/voided/refunded (lines with batch and qty)	Sale deductions (Section 6.2)

3. Schema

Restated from doc 18 v0.2 part B (M5 tables) and doc 25 §3 (CR-25-1). Location db/migration/m5inventory/.

```
-- V0001_inventory.sql (RLS per 17A 6.3 on every table; grants: ledger INSERT only)
CREATE TABLE inventory.stock_lot (
  stock_lot_id uuid PRIMARY KEY, location_id uuid NOT NULL, batch_id uuid NOT NULL,
  condition text NOT NULL DEFAULT 'GOOD' CHECK (condition IN ('GOOD','DAMAGED')),
  qty_on_hand numeric(14,3) NOT NULL DEFAULT 0, unit_cost numeric(14,4) NOT NULL CHECK (unit_cost >= 0),
  received_at timestamptz NOT NULL, negative_since timestamptz, negative_acknowledged_at timestamptz,
  last_movement_seq bigint NOT NULL DEFAULT 0, owner_entity_id uuid NOT NULL,
  UNIQUE (location_id, batch_id, condition));
CREATE INDEX lot_by_location_batch ON inventory.stock_lot (location_id, batch_id) WHERE qty_on_hand <> 0;

CREATE TABLE inventory.stock_movement (
  movement_id uuid NOT NULL, received_at timestamptz NOT NULL DEFAULT now(),
  source text NOT NULL, movement_seq bigint NOT NULL, -- dense per (location, source)
  location_id uuid NOT NULL, batch_id uuid NOT NULL, condition text NOT NULL,
  qty_delta numeric(14,3) NOT NULL CHECK (qty_delta <> 0),
  movement_type text NOT NULL CHECK (movement_type IN
    ('RECEIPT','SALE','SALE_REVERSAL','TRANSFER_OUT','TRANSFER_IN',
    'REPACK_CONSUME','REPACK_PRODUCE','COUNT_ADJUST','WRITE_OFF','RETURN_TO_SELLER','OPENING_BALANCE','GRN_REVERSAL')),
  unit_cost_at_movement numeric(14,4) NOT NULL, document_id uuid NOT NULL, document_line_id uuid,
  occurred_at timestamptz NOT NULL, occurred_local timestamp, operator_user_id uuid, device_id uuid,
  owner_entity_id uuid NOT NULL,
  PRIMARY KEY (received_at, movement_id), UNIQUE (received_at, location_id, source, movement_seq)
) PARTITION BY RANGE (received_at);
CREATE INDEX mv_by_lot ON inventory.stock_movement (location_id, batch_id, received_at);
CREATE INDEX mv_by_doc ON inventory.stock_movement (document_id);
GRANT INSERT ON inventory.stock_movement TO app_rw; -- no UPDATE, no DELETE (doc 18 P4)

CREATE TABLE inventory.entity_sku_cost (owner_entity_id uuid NOT NULL, sku_id uuid NOT NULL,
  qty_on_hand numeric(14,3) NOT NULL DEFAULT 0, avg_cost numeric(14,4) NOT NULL DEFAULT 0, updated_at
  timestamptz NOT NULL,
  PRIMARY KEY (owner_entity_id, sku_id));

CREATE TABLE inventory.repack_recipe (recipe_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL, name text NOT
  NULL,
  input_sku_id uuid NOT NULL, input_qty numeric(14,3) NOT NULL, output_sku_id uuid NOT NULL, output_qty
  numeric(14,3) NOT NULL,
  expected_loss_pct numeric(5,2) NOT NULL DEFAULT 0, status text NOT NULL DEFAULT 'ACTIVE' CHECK (status IN
    ('ACTIVE','RETIRED')),
  CHECK (input_sku_id <> output_sku_id));

CREATE TABLE inventory.loss_category (code varchar(24) PRIMARY KEY, name_en text NOT NULL, name_si text,
  name_ta text, photos_required boolean NOT NULL DEFAULT false);
CREATE TABLE inventory.variance_threshold (threshold_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL,
  sku_id uuid, tag_code varchar(40),
  tolerance_pct numeric(5,2), tolerance_qty numeric(14,3), band1_limit numeric(14,2), band2_limit
  numeric(14,2),
  CHECK ((sku_id IS NULL) <> (tag_code IS NULL)));
CREATE TABLE inventory.loss_tolerance (tolerance_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL, sku_id
  uuid, tag_code varchar(40),
  pct numeric(5,2), qty numeric(14,3), CHECK ((sku_id IS NULL) <> (tag_code IS NULL)));
CREATE TABLE inventory.expiry_window (window_id uuid PRIMARY KEY, owner_entity_id uuid, sku_id uuid, tag_code
  varchar(40), days smallint NOT NULL);

CREATE TABLE inventory.count_task (task_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL, location_id uuid
  NOT NULL,
  scope_kind text NOT NULL CHECK (scope_kind IN ('FULL','SKUS','TAGS','EXPIRY')), scope jsonb NOT NULL,
  scheduled_for date NOT NULL,
  status text NOT NULL DEFAULT 'SCHEDULED' CHECK (status IN
    ('SCHEDULED','COUNTING','SUBMITTED','VARIANCE_REVIEW','CLOSED')),
  expectation jsonb, started_at timestamptz, count_document_id uuid, adjustment_document_id uuid,
  expiry_task_id uuid);
CREATE TABLE inventory.expiry_task (task_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL, location_id uuid
  NOT NULL, batch_id uuid NOT NULL,
  days_to_expiry smallint NOT NULL, qty_at_open numeric(14,3) NOT NULL, status text NOT NULL DEFAULT 'OPEN',
  count_task_id uuid, suggested_writeoff_id uuid);
CREATE TABLE inventory.pick_list (pick_list_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL,
  delivery_document_id uuid NOT NULL,
  lines jsonb NOT NULL, overrides jsonb NOT NULL DEFAULT '[]', status text NOT NULL DEFAULT 'OPEN');

-- document extensions (partitioned like the base): doc_count_line(line_id, expected_qty, counted_qty,
  skipped, skip_reason);
-- doc_adjustment(document_id, from_count_document_id, value, band, approver_user_id);
-- doc_writeoff(document_id, category, value,
```

```
-- witness_user_id, witnessed_at, approver_user_id, remote_witness boolean); doc_repack(document_id,
  recipe_id, input_lot_ids, actual_output_qty,
-- expected_output_qty, variance_qty, output_batch_id, reversal_of); doc_transfer(document_id,
  from_location_id, to_location_id,
-- transfer_request_id, dispatched_at, received_at, received_by); doc_opening_balance(document_id,
  signed_entity_by, signed_entity_at,
-- countersigned_by, countersigned_at)
```

3.1 Seed data

```
# seed/m5/loss-categories.yaml
- {code: DAMAGED_IN_TRANSIT} - {code: DAMAGED_IN_STORE} - {code: EXPIRED} - {code: THEFT, photos_required:
  true}
- {code: SHRINKAGE_UNEXPLAINED, photos_required: true} - {code: STAFF_CONSUMPTION} - {code: SAMPLES} - {code:
  DONATION} - {code: OTHER}
# seed/m5/config.yaml  inventory.transfer_receipt_days 3 | inventory.remote_witness_allowed true |
  inventory.writeoff_photos_required [THEFT, SHRINKAGE_UNEXPLAINED, single_staff]
# seed/m5/defaults.yaml  federation defaults by governed tag: expiry windows (dairy 7, fresh-produce 3,
  dry-goods 30), tolerance 1% or 2 units,
#
  bands 25,000 / 250,000
# seed/m5/permissions.yaml (appended): inv.count.schedule (ENTITY); shop.count.record (LOCATION, offline);
  inv.adjust.request (ENTITY);
# inv.adjust.approve (ENTITY, mfa, limits max_value); inv.writeoff.request (LOCATION, offline draft);
  inv.writeoff.witness (ENTITY);
# inv.writeoff.approve (ENTITY, mfa, limits max_value); inv.recipe.manage (ENTITY); inv.repack.execute
  (LOCATION, offline); inv.repack.reverse (ENTITY);
# inv.transfer.issue (ENTITY); shop.transfer.receive (LOCATION, offline); inv.opening.prepare (ENTITY);
  inv.opening.sign (ENTITY, mfa);
# inv.opening.countersign (F, mfa); inv.policy.set (ENTITY); whs.pick (ENTITY)
```

4. Package layout

```
lk.coopfed.knoweb.m5inventory/  @ApplicationModule(allowedDependencies =
  {"kernel","m1party::api","m1party::query","m2catalogue::api","m2catalogue::query"})
api/      commands, events, value types (Movement, LotRef, Band)
query/    InventoryQueries (balances, availability, inStockBatches, lotsConsumed, hasAnyLot, skusWithLots,
  pickBatches, entityAverageCost, expiring, lotSnapshot),
  LossQueries (lossSummary, shrinkagePct)
internal/
  ledger/      LedgerService (post), LotRepository, MovementRepository, CostService (weighted average),
    SequenceService (per location+source)
  availability/ AvailabilityService, FefoService, PickListService
  count/       CountTask, handlers, ExpectationSnapshot, VarianceEvaluator (tolerances, bands)
  adjust/      Adjustment handlers
  writeoff/    WriteOff handlers, WitnessRules
  repack/      Recipe, Repack handlers, YieldVariance
  transfer/    Transfer handlers
  opening/     OpeningBalance handlers
  expiry/      ExpiryScanJob, ExpiryTaskService
  consumers/   GrnConsumer, ReceiptConsumer (sale deductions), ClaimReturnConsumer, DeliveryConsumer (pick
    lists, reservations), BatchCorrectionConsumer, TransferRequestConsumer
  jobs/        StockReconcileJob, TransferOverdueJob, NegativeLotReviewJob
  snapshot/    LotSnapshotContributor, CountTaskSnapshotContributor, PolicySnapshotContributor
  ingest/      CountBundleHook, WriteOffDraftHook, RepackBundleHook, TransferReceiptHook
web/          StockController, CountController, AdjustmentController, WriteOffController, RepackController,
  TransferController, OpeningController, PolicyController, PickController
```

5. API slice: openapi/m5inventory.yaml (excerpt)

```
/v1/inventory/locations/{id}/balances    get (skuId?, tag?, includeZero?) -> [{skuId, batchId, batchNo,
    expiry, condition, qtyOnHand, fefoRank, unitCost? (OWN only)}]
/v1/inventory/availability               get (locationIds[], skuIds[]) -> [{locationId, skuId,
    available}]
/v1/inventory/skus/{id}/movements        get (locationId?, from, to, cursor)
/v1/inventory/counts                     post scheduleCount {locationId, scopeKind, scope, scheduledFor}
    get listCounts
/v1/inventory/counts/{taskId}/start|submit|skip-line post
/v1/inventory/adjustments                 post createAdjustment /{id}/submit|approve|reject post
/v1/inventory/write-offs                  post requestWriteOff {locationId, category, lines[{batchId,
    condition, qty}], attachmentIds[]}
/v1/inventory/write-offs/{id}/submit|witness|approve|reject post
/v1/inventory/recipes                     post | get /{id}/retire post
/v1/inventory/repacks                     post planRepack {recipeId, inputLotIds[], locationId}
    /{id}/execute {actualOutputQty} | /reverse {reason} post
/v1/inventory/transfers                   post issueTransfer {fromLocationId, toLocationId, lines[],
    transferRequestId?} /{id}/dispatch|receive|cancel post
/v1/inventory/opening-balances             post prepare /{id}/sign|countersign post
/v1/inventory/policies                     put setVarianceThreshold | setLossTolerance | setExpiryWindow get
/v1/inventory/pick-lists/{dnId}           get /lines/{lineId}/override post
/v1/inventory/expiry-tasks                 get (locationId, status)
/v1/inventory/loss-summary                 get (scope, period) /shrinkage get (locationId, period)
/v1/inventory/lots/{id}/acknowledge-negative post
# example - balances for a shop (till-scoped: no unitCost)
[ { "skuId": "...", "batchId": "...", "batchNo": "B2411A", "expiry": "2027-03-31", "condition": "GOOD",
    "qtyOnHand": 37.000, "fefoRank": 1 },
  { "skuId": "...", "batchId": "...", "batchNo": "B2503C", "expiry": "2027-06-30", "condition": "GOOD",
    "qtyOnHand": 48.000, "fefoRank": 2 } ]
```

6. The ledger core and handler specifications

6.1 LedgerService and CostService

```
// ledger/LedgerService.post(List<Movement> ms, Document doc, ScopeContext ctx) -- the only writer of
    stock_lot.qty_on_hand
@Transactional
void post(ms, doc, ctx) {
    for (m : ms) {
        Lot lot = lots.lockOrCreate(m.locationId, m.batchId, m.condition, defaultCost = m.unitCost ? :
            cost.avg(ctx.entityId, skuOf(m.batchId))) // SELECT ... FOR UPDATE
        long seq = sequences.next(m.locationId, source = ctx.isDevice ? m.deviceSeqSource : "central")
        // dense per (location, source)
        movements.insert(m.withSeq(seq).withCost(costFor(m, lot, ctx)).citing(doc))
        lot.qtyOnHand += m.qtyDelta; lot.lastMovementSeq = seq
        if (lot.qtyOnHand < 0 && lot.negativeSince == null) { lot.negativeSince = now;
            events.publish(LotNegative.of(lot, doc)); audit REVIEW NEGATIVE_STOCK }
        if (lot.qtyOnHand >= 0 && lot.negativeSince != null) { lot.negativeSince = null; audit INFO
            NEGATIVE_CLEARED }
        lots.save(lot)
        cost.apply(m, ctx.entityId)
        // below
        events.publish(StockMoved.of(m, lot, doc))
    }
}
costFor(m, lot, ctx) = switch (m.type) { RECEIPT, OPENING_BALANCE, REPACK_PRODUCE -> m.unitCost; TRANSFER_IN
    -> m.unitCost (source lot cost);
    default (issues: SALE, WRITE_OFF, REPACK_CONSUME, COUNT_ADJUST-,
        TRANSFER_OUT, RETURN) -> cost.avg(entity, sku) }
// ledger/CostService.apply(m, entity): moving weighted average per (entity, sku) -- doc 25 3.3
row = costs.lock(entity, sku)
if (m.isIntake() && m.type != TRANSFER_IN) { row.avg = (row.qty * row.avg + m.qty * m.unitCost) / (row.qty +
    m.qty); row.qty += m.qty }
else if (m.type == TRANSFER_IN) { row.qty += m.qty } // within the entity: no cost change
else { row.qty += m.qtyDelta } // issues change qty, not avg
// avg never negative; if row.qty <= 0 after an issue, avg is kept (last known) - property test asserts
```

6.2 Consumers (the apply commands)

```
@EventConsumer(types={"grn.confirmed.v1"}, consumer="m5.receipts") // doc 24 6.1 payload
    for line: post RECEIPT (location, batch, GOOD, received - damaged, cost=line.unitCost) and RECEIPT (DAMAGED,
        damaged) when > 0
@EventConsumer(types={"grn.reversed.v1"}) post GRN_REVERSAL negating each original movement (guarded
    upstream by LotsConsumed)
@EventConsumer(types={"receipt.issued.v1","receipt.refunded.v1","receipt.voided.v1"}, consumer="m5.sales")
    // M6 bundles via ingestion
```

```

issued:  for line with qty>0: post SALE (location, line.batchId, GOOD, -qty) -- the batch the till
         resolved; negative allowed (doc 25 3.2)
         if no lot exists for (location, batch): lots.create with qty 0 and cost = avg; audit REVIEW
         SALE_WITHOUT_LOT (DR-5)
         refunded/voided: post SALE_REVERSAL (+qty) against the same lot
@EventConsumer(types={"claim.approved.v1"}) when returnRequired: post RETURN_TO_SELLER (-qty) from the claim's
         condition lot
@EventConsumer(types={"drop.undelivered.v1"}) post RECEIPT back into the seller's dispatching location
         (returned goods)
@EventConsumer(types={"delivery_note.issued.v1"}) PickListService.create(dn): FEFO suggestions per line;
         reservation = issued-undispatched qty for availability
@EventConsumer(types={"transfer_request.approved.v1"}) IssueTransfer(from, to, lines) then publish
         transfer.issued.v1 (M4 writes back the id)
@EventConsumer(types={"batch.corrected.v1"}) lots.repoint(oldBatch -> newBatch) in place (qty and cost
         unchanged); audit INFO

```

6.3 User command specifications

Command	Guards (in order)	Effect	Events
ScheduleCount	entity scope; location in scope; scope valid	count_task SCHEDULED; change-log to the location (task row)	count.scheduled.v1
StartCount	location scope; SCHEDULED; expectation snapshot = lots in scope now	COUNTING	count.started.v1
SubmitCount	COUNTING; every scope line counted or skipped with reason; DocumentService.issue (LOCATION series); expectation adjusted by movements since start (E-06)	SUBMITTED; VarianceEvaluator: in-tolerance variances post COUNT_ADJUST now; others form ADJ DRAFT → VARIANCE_REVIEW; else CLOSED	count.submitted.v1; adjustment.created
CreateAdjustment / SubmitAdjustment	entity scope; lines with lots; reason	ADJ issued (ENTITY series); value = $\sum qty \times \text{avg cost}$; band from thresholds	adjustment.submitted.v1
ApproveAdjustment	PENDING; Sod.assertDistinct(request, approve, requester); approver limit $\geq$ value; MFA	APPROVED → COUNT_ADJUST movements → POSTED; count task CLOSED	adjustment.approved/ posted.v1
RejectAdjustment	reason	REJECTED; count CLOSED	adjustment.rejected.v1
RequestWriteOff	location scope; lot exists; qty $\leq$ on hand or lot negative-clearing; category	DRAFT (offline allowed)	–
SubmitWriteOff	DRAFT; photos referenced when category requires or location single-staff; issuance (LOCATION or ENTITY series)	REQUESTED	writeoff.requested.v1
WitnessWriteOff	REQUESTED; witness $\neq$ requester; attachments COMPLETE; remote witness allowed only when the location is single-staff (config) and the witness holds approve	WITNESSED (remote_witness flag)	writeoff.witnessed.v1
ApproveWriteOff	WITNESSED; approver $\neq$ requester; value ($\sum \text{qty} \times$ avg cost) $\leq$ approver band; MFA	APPROVED → WRITE_OFF movements → POSTED	writeoff.approved/pos ted.v1
RejectWriteOff	reason	REJECTED	writeoff.rejected.v1
DefineRecipe / RetireRecipe	input $\neq$ output; both SKUs active; output origin REPACK_OUTPUT (M2)	row	recipe.*.v1
PlanRepack / ExecuteRepack	recipe ACTIVE; input lots GOOD with qty $\geq$ recipe input; issuance; consume/produce movements; output cost = consumed cost / actual output; variance = expected – actual; output batch via M2 RegisterBatch (expiry from input batch)	EXECUTED	repack.executed.v1; stock.moved.v1
ReverseRepack	EXECUTED; output lot qty = produced and no movements against it; reason	REVERSES-linked RPK: negating movements; input lot restored	repack.reversed.v1
IssueTransfer / Dispatch / ReceiveTransfer / CancelTransfer	same entity; availability at source; issuance; TRANSFER_OUT at cost; destination receive posts TRANSFER_IN with actuals (shortfall → ADJ draft at destination); cancel before receipt negates	per state	transfer.*.v1

Command	Guards (in order)	Effect	Events
PrepareOpeningBalance / Sign / Countersign	location ONBOARDING with no movements; lines with batch data and cost; entity officer MFA; Federation officer MFA; issuance; OPENING_BALANCE movements	POSTED	opening_balance.posted.v1
SetVarianceThreshold / SetLossTolerance / SetExpiryWindow	entity scope (Federation for governed-tag defaults)	rows; policy snapshot change	inventory_policy.changed.v1
OverridePick	warehouse scope; reason	pick_list.overrides	pick.overridden.v1
AcknowledgeNegativeLot	entity scope; note	acknowledged_at	lot.acknowledged.v1

7. Queries, jobs, snapshot and till bundles

Item	Implementation
Availability	Σ GOOD lots with qty_on_hand > 0 at the locations – Σ pick_list reserved (dn ISSUED not DISPATCHED); per (location, sku); cached per instance 30 s; PARTY callers get the seller's configured mode (live or the last end-of-day figure).
InStockBatches	GOOD lots with qty > 0 joined to M2 batch (printed MRP, expiry) for M3's authoring checks.
LotsConsumed(grn)	Any movement other than the GRN's own RECEIPT (and GRN_REVERSAL) against the lots the GRN created.
FefoService	Rank GOOD lots by expiry nulls-last, then received_at; recomputed on change for the snapshot.
ExpiryScanJob (nightly)	For each (location, batch) with qty > 0 and expiry within the window for the SKU or tag (most specific wins): open expiry_task, create an EXPIRY-scope count task, publish expiry_task.opened.v1; on expiry date passed: create a suggested WOF DRAFT (category EXPIRED) for the remaining qty.
StockReconcileJob (nightly)	Per location: recompute Σ movements per lot and compare with qty_on_hand; drift → audit REVIEW STOCK_DRIFT with both figures; never corrects (doc 25 3.2).
TransferOverdueJob (daily)	IN_TRANSIT longer than transfer_receipt_days → REVIEW.
NegativeLotReviewJob (daily)	Lots negative longer than 7 days without acknowledgement → ALERT to the entity.

7.3 Snapshot contributors

```

LotSnapshotContributor: tables(): ["lot"]
  rowsFor(location): GOOD and DAMAGED lots with qty <= 0 -> {stockLotId, batchId, skuId, condition, qtyOnHand,
    fefoRank, negative: bool} // no cost
  batch ids supplied to M2's contributor so batch rows (batchNo, expiry, printedMrp) travel with them (doc 22A
  7.3)
  change-log: every stock.moved.v1 for the location -> UPSERT lot row (coalesced per 5 minutes by the worker
    fan-out to limit churn)
CountTaskSnapshotContributor: tables(): ["count_task"] rowsFor(location): SCHEDULED/COUNTING tasks with scope
  and expectation
PolicySnapshotContributor: tables(): ["inventory_policy"] rowsFor(location): tolerances by sku/tag; loss
  categories; photos_required; expiry windows
ExpectedTransferContributor: tables(): ["expected_transfer"] rowsFor(location): IN_TRANSIT transfers to this
  location with lines

```

7.4 Till bundles (contract with M6)

```

count.submitted.v1 (bundle): document(CNT, LOCATION series) + lines[{lineId, skuId, batchId, condition,
  expectedQty, countedQty, skipped, skipReason}] + task id
hook: CountBundleHook -> VarianceEvaluator as in SubmitCount; the till's expectation is trusted for lines
  counted offline; central recomputes and REVIEWS a difference
writeoff.draft.v1 (bundle): document(WOF, DRAFT, unnumbered) + lines + category + attachments (PENDING
  allowed)
hook: WriteOffDraftHook -> stored as DRAFT; SubmitWriteOff happens online (H-03: approval-path documents
  issue online)
repack.executed.v1 (bundle): document(RPK, LOCATION series) + recipe id + input lot ids + actualOutputQty
hook: RepackBundleHook -> movements as ExecuteRepack; output batch registered; variance recorded
transfer.received.v1 (bundle): document(XFR receipt, LOCATION series) + transfer id + lines[{skuId, batchId,
  receivedQty}]
hook: TransferReceiptHook -> TRANSFER_IN; shortfall -> ADJ draft at destination
sale deduction: not a separate bundle - M6's receipt bundle lines carry batchId; m5.sales consumes
  receipt.issued.v1 (6.2)

```

8. Screens

```

Stocktake sheet (till, shop-management mode) - specification for M6
+-----+
| ScopeBanner: Shop 07 | shop mode | shop-in-charge | OFFLINE |
| Count: Dairy (weekly) scheduled 2026-10-13 started 09:12 12 of 18 counted |
| [thumb] Milk powder 400g B2411A exp 2027-03 expected 37 counted [ 35 ] !-2 |
| [thumb] Milk powder 400g B2503C exp 2027-06 expected 48 counted [ 48 ] |
| [thumb] Yoghurt 500ml Y0921 exp 2026-10-15 expected 12 counted [ ] [skip: reason v] |
| variance shown only after entry; tolerance-cleared lines marked; [Submit count] (all lines counted or
  skipped) |
+-----+
Damage/expiry register (write-off request, till or web): category picker with icons (expired, damaged,
  theft...), lines with lot picker
  (FEFO order), qty, camera strip (required badge when policy says so), [Save draft] (offline) [Submit]
  (online) -> shows who must witness/approve
Adjustment voucher (web, approver): lines with expected/counted/variance/value; band and my limit; [Approve]

```

-> StepUpDialog; [Reject] -> ReasonCapture

Screen	Components	Message ids (m5.)	Notes
Stock book / stock card	list with ItemCard rows, expiry badges, negative flag; Timeline of movements	stock.*	Cost column only for OWN back office
Stocktake sheet	as sketched (M6 renders from the count_task snapshot row)	count.*	Expectation adjusted during count
Adjustment voucher	DocumentLines, band panel, ApprovalBar, StepUpDialog	adjust.*	SoD message when requester = approver
Damage/expiry register	category grid, lot picker, AttachmentCapture, ApprovalBar with the next required actor	writeoff.*	Remote-witness badge for single-staff shops
Production sheet	recipe picker, input lots, actual yield, variance, [Reverse] while eligible	repack.*	–
Transfer note	issue and receive forms; expected lines at destination from snapshot	transfer.*	–
Pick list	per DN line: suggested lots; override with reason	pick.*	Warehouse only
Expiry list	tasks with days-to-expiry, qty, markdown rule from M3, suggested write-offs	expiry.*	Web and till
Opening stock sheet	lines with batch data and cost; two signature steps with StepUpDialog	opening.*	–
Loss and shrinkage	by category and location; tolerances; Federation view	loss.*	–
Policies	thresholds, tolerances, expiry windows by sku/tag	policy.*	–

9. Tests

Layer	Cases	Target
Unit	Every guard; CostService arithmetic through doc 13 scenario 3 (9,500/49 = 193.88) and mixed receipts; FEFO ranking with null expiry; availability with reservations; VarianceEvaluator tolerances and bands; yield variance; witness rules incl. remote	All branches
Property-based (jqwik)	Lot balance = Σ movements under random interleavings incl. reversals and negatives; avg cost equals recomputed-from-scratch and never negative; ReverseRepack restores the input lot exactly; availability never includes negative lots; adjustments from a count never double-post in-tolerance lines	5,000 cases each
Contract	Every operation; till-scoped balances carry no cost; PARTY availability respects the seller's mode	Every operation
State machine	All seven machines incl. offline transitions via bundle hooks	All transitions
Consumers	grn.confirmed creates GOOD and DAMAGED lots; grn.reversed negates; receipt lines deduct the resolved batch; oversell goes negative with lot.negative; claim return; undelivered drop returns to stock; transfer request issues XFR	Seven cases
Scenario	Doc 13 scenarios 3 and 4 (M5 parts); flows 6.1–6.9 of doc 25; two till simulators overselling the same last units	Ten flows
Reconciliation	Injected drift found and reported, not fixed; NegativeLotReviewJob escalates after 7 days	Two cases
Performance	1,000 receipt bundles/s sustain balance updates on the two-instance stack; reconcile 10,000 locations under 30 min; expiry scan under 10 min	Three targets
Web	Playwright: schedule count → (simulated till submit) → adjustment approve with step-up; write-off request → witness → approve; repack execute → reverse	Three flows

10. Tickets in build order

#	Ticket	Est. (days)	Done when
M5-01	Scaffold; migrations (lot, movement partitioned, cost, policies, tasks, extensions); seeds; RLS	2	Applies; grants tested (movement INSERT only)
M5-02	LedgerService, SequenceService, CostService; property tests	3	Balance and cost properties green
M5-03	GrnConsumer, ReceiptConsumer (sales, oversell), ClaimReturn, DeliveryConsumer (pick lists, reservations), BatchCorrection consumers	3	Consumer tests; two-simulator oversell
M5-04	Availability, FEFO, InStockBatches, LotsConsumed, HasAnyLot, SkusWithLots, EntityAverageCost queries; replace the M2/M3/M4 stubs	1.5	Dependants' tests pass against real queries
M5-05	Counts: task, start with expectation snapshot, submit with adjustment during count, VarianceEvaluator, CountBundleHook	3	Flow 6.3 incl. offline bundle
M5-06	Adjustments with bands, SoD, MFA; NegativeLotReview job	1.5	Approval matrix tests
M5-07	Write-offs: full lifecycle, witness rules incl. remote, photos policy, WriteOffDraftHook	2.5	Flow 6.4
M5-08	Recipes, repack execute and reverse, output batch registration, RepackBundleHook	2	Scenario 3
M5-09	Transfers issue/dispatch/receive/cancel; TransferRequestConsumer; TransferReceiptHook; overdue job	2	Flow 6.6
M5-10	Opening balances with two signatures; M1 activation gate event	1	Flow 6.9
M5-11	ExpiryScanJob and tasks; StockReconcileJob; policies; snapshot contributors and change-log fan-out	2.5	Flow 6.8; reconcile test
M5-12	Web screens (eleven) and till screen specs; Playwright flows	3.5	Three flows green
M5-13	Contract freeze: m5inventory.yaml v1; bundle contracts signed by the till track; api/query review with M6, M8; doc 33 slice	0.5	M6 acknowledges

Total about 28 developer-days. Tickets M5-02 to M5-04 replace the stubs that M2, M3 and M4 have been building against; run their suites as part of M5-04's definition of done.

11. Open points: what to assume, whom to ask

Point	Assume	Decider	If different
Availability definition, damaged lots, write-off value basis, remote witness, sale-without-lot, opening balance gate (doc 25 DR-1 to DR-6)	As proposed in doc 25 §11	Federation admins; architecture review	Config or one-line guard changes
New tables (DR-7)	Adopted; CR-25-1 raised	Architecture review	–
Count-while-trading expectation (DR-8)	Movements during the count adjust the expectation; the till applies the same rule locally	Architecture review, till track	Freeze expectation at start and report all sales as variance (worse)
Movement sequence source column	Dense per (location, source) with source = device id or 'central'	Kernel	–
Snapshot churn from sales	Lot rows coalesced per 5 minutes in the worker fan-out	DevOps after spike 5	Tune the window
Single-staff detection	Location size band S or config flag	M1 developer	Add an explicit staff-count attribute

End of document. 25A v0.1; m5inventory/README.md and its tests become the living version once merged.